

Crop Recommendation System

Areesha Faisal, Maryam Sana, Mishaal Hassan

Department of Computer Science

Kinnaird College for Women University

Abstract

This research has developed a system to recommend crops, which takes into account both the climatic and soil conditions of the field and generates one of 57 possible crop recommendations. The seven input variables consist of nitrogen, phosphorus, potassium, temperature, relative humidity, pH levels in the soil, and the amount of rainfall received; these seven inputs are used by a decision tree model to calculate a single result out of a possible 57 crops. Because the decision tree was trained on 57,000 examples of similar inputs and outputs, this crop recommendation system may provide a highly accurate estimate of the type of crop that should be planted in the user's field. The decision tree classified all input variables within a flask-based web service that has an application programming interface (API) for prediction; the user inputs variable values using React.js on a web page that displays an immediate result when the user has entered their variable values into the web page. Additionally, chat functionality will provide the user with more information about the dataset, creation of the dataset, decision tree reasoning, how the crop recommendation system is set up and accuracy of all data used by the system.

Introduction

There are numerous variables to consider when selecting the type of crop to plant, including soil nutrient levels (e.g., nitrogen, phosphorus, potassium), weather conditions (temperature, humidity, etc.), soil type (acidic, alkaline, etc.), and how much rainfall there may be in each growing season. Selecting a crop often relies on the gut instinct or historical experience of the producer, resulting in a poor yield if the expected growing conditions do not match the actual growing condition for a crop to be planted.

Therefore, as part of this project, we created a simple tool that allows farmers to enter the above-mentioned 7 environmental parameters (variables) into the tool and obtain an answer as to which type of crop would be the best fit for those environmental parameters. By using this tool in place of detailed agricultural science knowledge, farmers can now use their own work experience and the data collected by other farmers to assist them in determining which crops they should grow.

We will discuss the dataset(s) that we utilized, how we created features to correspond with our model, our reasoning for choosing Decision Trees over other algorithms, and how everything from the model to the end-user web interface will be connected.

Dataset

We acquired 57000 crop records from Kaggle which represent the 57 variety of crops recorded as well as 1000 replicates for each crop. Every crop has been evenly distributed with 1000 of all replicates hence no single crop will use its total amount of replicates. There are several different vegetable (Toma, Eggplant, Pepper) , cash crops (Sugar Cane, Cotton) , Cereal (Rice, Wheat, Corn) , pulse (Black Gram, Green Gram) and oil seed crops (Groundnut, Sunflower) included in this dataset along with other various types of crops on the records. Rather than being stored as two numbers (for example - N for Nitrogen could have low value and high value) vegetation's have been coded from their original variable measurement per input into the database and then based on whatever physical characteristics will be subjectively applied. Each crop will share fixed data points i.e; Nitrogen (0 - 20, 1 - 20, 2 - 20 etc) not real numbers.

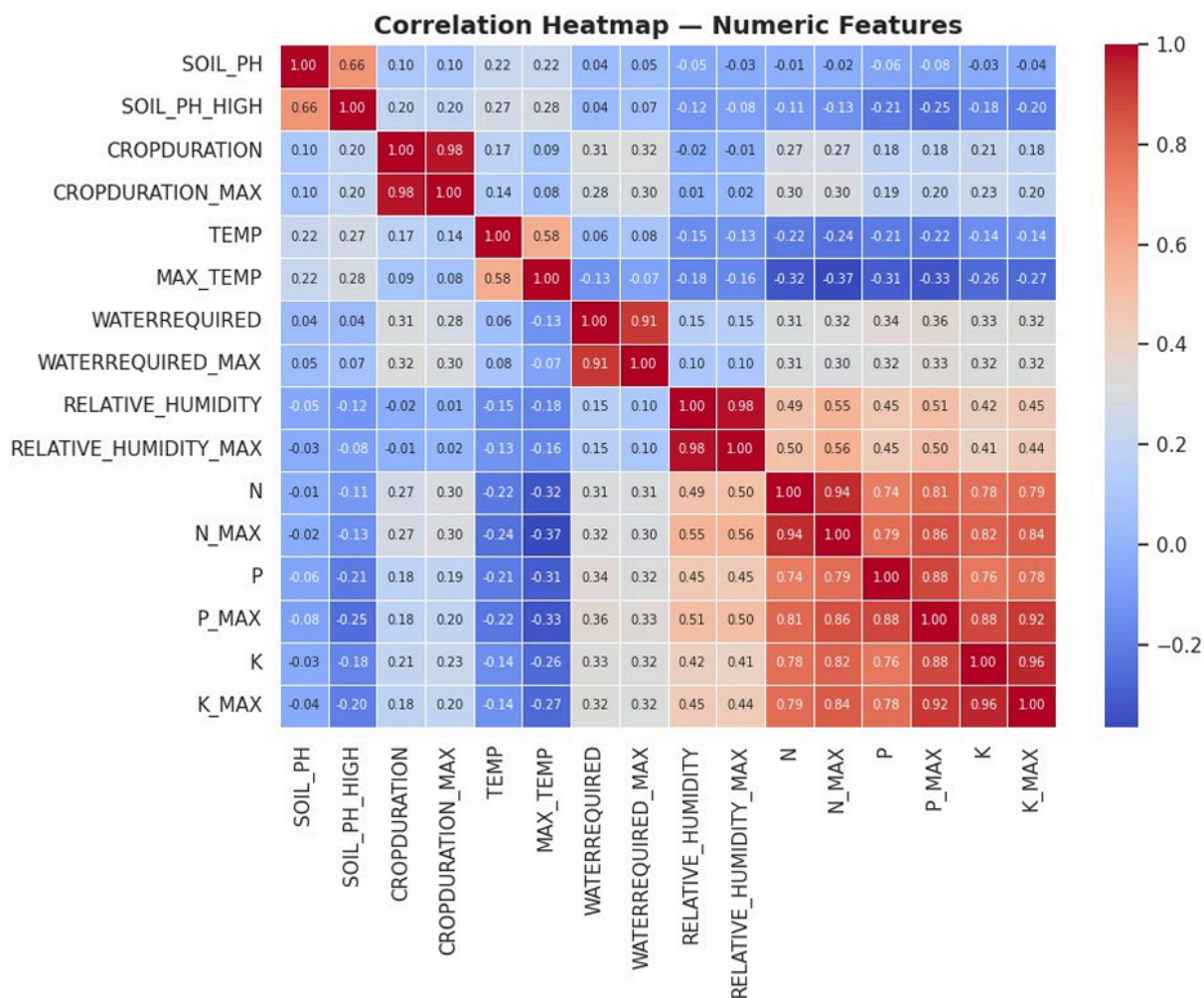
Link: <https://www.kaggle.com/datasets/saganachinnathambi/crop-recommendation/data>

A. Averaging Min and Max

For each of the 7 parameters:

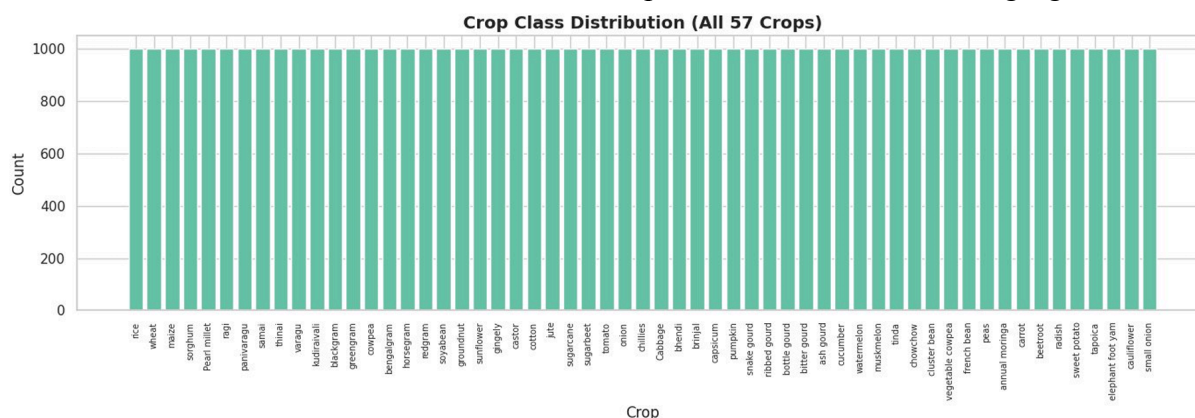
$$\text{value} = \frac{\text{min} + \text{max}}{2}$$

We created one single numerical value or quantity by halving both of the temperatures; i.e., there is one number for each crop feature; hence, just as a user of software would input temperature as an integer (i.e., not a range) we have now calculated a correlation between the two temperature values. As shown in figure 1, we have also demonstrated that when we have the minimum and maximum temperature values for each crop feature, they will both be strongly correlated (will always have a correlation of 0.94-0.98) and therefore averaging them together will not negate or substantially diminish any relative importance from either of the individual datasets.



B. Class Balance

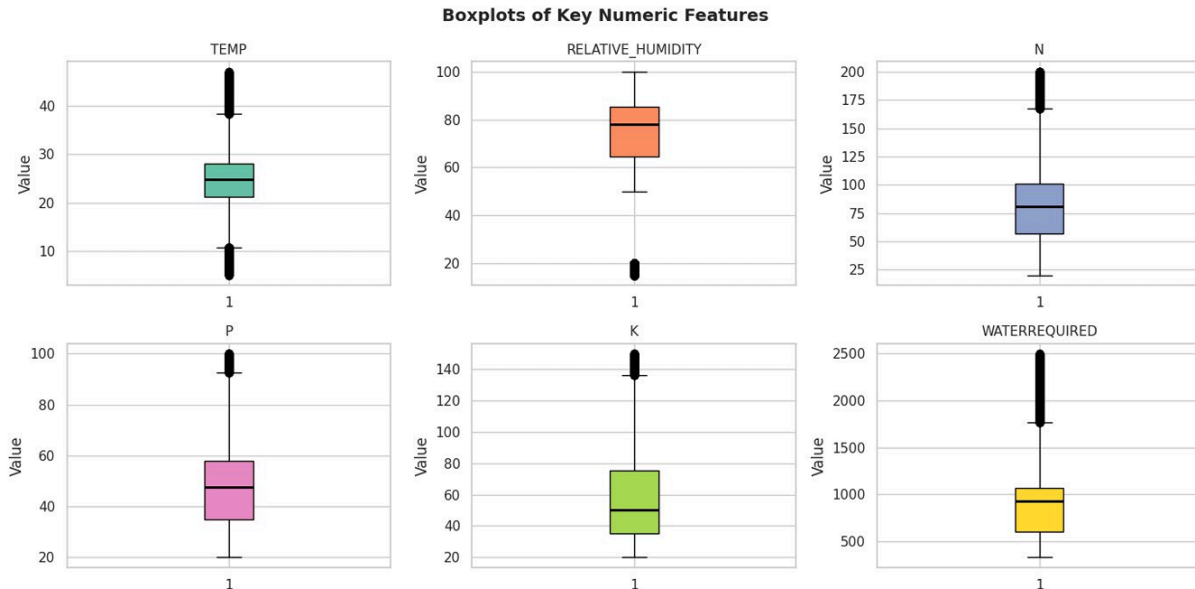
Perfect balance across all 57 classes of crops, as shown in the following figure:



C. Feature Ranges

In Box Plots we examined each of the key numerical variables to see how large or small they could vary (Fig 3). The temperature appears to generally range from 20-28 degrees Celsius, while the humidity appeared to be in the general range of 65%-85%,

but the amount of water used by the different types of crops has a much larger variability. Most crop types within the data set appear to require less than 1000 mm of water per crop type, while there were 2 crops (rice and sugarcane) that could use more than 2500 mm of water per crop type.



Methodology

A. Preparing the Data

We created a set of minimum and maximum MM features and developed a process which identifies seven consistent characteristics across the crops. Those characteristics are:

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Temperature
- Humidity
- Soil pH
- Rainfall (water threshold).

Because the target column (the output) contains text (e.g., rice, maize), Scikit-learn's label encoding was used to convert the text labels of the crops into numerical representations that were input into the model. The goal is then to use the label encoding to also associate an output from the prediction model back to a crop name, rather than guessing or coding it in programming to display at the end-user level.

B. Why a Decision Tree

We chose to utilize a Decision Tree classifier due to its practical benefits:

- Training time was minimal; fitting the model took only seconds instead of minutes, making the training iterations simple.
- There was no requirement for pre-scaling/normalization since the algorithm can be trained directly from the data set.
- The algorithm can support multiple class instances simultaneously, thus negating the need for additional separation (one-vs-rest) to train our model using the 57 cropped images.
- The rule splits can be traced back easily; therefore the modifiable feature, which has been used in the rule split, is known and can be viewed as to why the algorithm made its selected splits. This has greatly aided our efforts in debugging the model during the model's development, and was critical to us.

The Model was trained with DecisionTreeClassifier from the scikit-learn library, with Gini Impurity being the default parameter for splitting criteria. The trained model and label encoder were saved to .pkl files with Python's Pickle library for usage after server restarts, without the need to re-train either component.

C. Architecture

There are three main systems that make up this system and interact with each other:

1. The Model-best model.pkl-contains a trained Decision Tree and its corresponding label encoder (label encoder.pkl).
2. The Backend-is a Flask application, the Model is loaded only one time when the Flask application starts; there is one API endpoint: /predict; the API accepts a POST request with a JSON containing the seven input values, once the information has been received, it returns the predicted crop.
3. The Frontend- developed with the React Framework using Vite, provides a user-facing form that allows for the entry and submission of the seven input fields and to return the predicted crop.

The front-end uses Axios library to send the user's information to the back-end. The back-end will check if the 5 values exist and are within logical ranges (for example: soil pH should be a number between 5 and 9, and the total rainfall should be a number between 330mm and 2,500mm) before being sent to the model for prediction generation. Any value that is not within spec will be returned to the front-end with a corresponding error message.

If any of the values are not within spec, an error message will be returned to the front-end for display to the user. Once the model has generated a prediction based upon the input values, and the label encoder has converted the label back to an actual

crop name for use in the front-end display, the back-end will send the crop name back to the front-end via JSON for display to the user.

D. Retraining

We created a new program named `retrain.py`. It does all of the same steps as the first project: load the dataset, compute the average of the minimum and maximum column values, encode the labels, train the Decision Tree Model (DTM), and create a model file of the DTM.

With this modification, if we ever have new records added to our Dataset, we can run `retrain.py` again, and it will overwrite the previous Model File, allowing us to use the updated Model without changing anything in either our front or back end.

Results & Discussion

The Decision Tree model performed best using this dataset, primarily due to the seven average features producing well-defined groups of crops that were very well-separated from each other. Furthermore, the data in this dataset is clean and based on simulated agronomic ranges which are easier to quantify than messy sensor datasets (i.e., actual sensor data), giving the Decision Tree model the ability to produce an extremely high level of accuracy on the train segment of the dataset.

The factors used to separate the crops in the Decision Tree were first and foremost the amount of water required to grow the crops and the soil pH level of the land to grow the crops. This is reasonable from an agronomy perspective because rice and sugarcane will have much different amounts of water required compared to crops grown on dry land, such as sorghum and chickpeas. The nitrogen and potassium levels in the soil were also used as a measure to determine the difference between cereal and pulse crops.

We found that the value-added backup of validation, offered by way of the system offered through a browser, has been useful. Prior to implementing our new validator into our web service; it would have been possible for someone to have uploaded a pH of 20, or attempted to upload a temperature of -50, and still received a model's response attempting to give the user an answer which does not conform with what actually happens in the real world. By creating this new validation system, we now have a means of preventing our users from being able to submit data that doesn't make any sense, thus providing users with a clear message as to why their input was rejected.

Along with the added benefit of our user's input being validated through the new backend validation system, the user experience with the new front end validation system was consistent and just as good. The form provides examples of typical values as placeholders, displays a loading state while the user is waiting for the backend response, and finally

displays either the predicted crops or an error message based upon what occurred with the request. The form does exactly what it was designed to do.

Conclusion & Future Work

As such this project serves as a demonstration that an easily understandable, straightforward decision tree model will produce truly usable crop recommendation tools provided that clean features and proper validation of the underlying data have taken place; thus allowing for usable tool development. Certainly, we need good datasets to use when preparing data pipelines or modelling crops (very important) and developing user interfaces (equally important); thus, there is a clear path forward to make this tool stronger via future steps, such as:

- Examine whether additional testing will improve random forest performance and/or accuracy through ensemble models.
- Provide options for confidence levels of either predicted outcomes or alternative possible prediction(s).
- Include the best possible prediction(s), as in the previous sentence; so users can view how reliable an actual predicted outcome(s) is.
- Link the project to soil sensors in order to minimize user manual entry of data.
- Develop the project for local farmers with support for Urdu and/or other regional languages.
- The redesign of the project front end should provide mobile compatibility or enable its use without the need to connect to the internet. Many farmers do not have continuous internet access.

AI Report (Quill Bot)

Abstract

This research has developed a system to recommend crops, which takes into account both the climatic and soil conditions of the field and generates one of 57 possible crop recommendations. The seven input variables consist of nitrogen, phosphorus, potassium, temperature, relative humidity, pH levels in the soil, and the amount of rainfall received; these seven inputs are used by a decision tree model to calculate a single result out of a possible 57 crops. Because the decision tree was trained on 57,000 examples of similar inputs and outputs, this crop recommendation system may provide a highly accurate estimate of the type of crop that should be planted in the user's field. The decision tree classified all input variables within a flask-based web service that has an application programming interface (API) for prediction; the user inputs variable values using React.js on a web page that displays an immediate result when the user has entered their variable values into the web page. Additionally, chat functionality will provide the user with more

🌐 English ▾ ⋮



quillbot Model Version: v7.1.0

0%

of text is likely AI ⓘ

AI-generated ⓘ

Human-written & AI-refined ⓘ

Human-written ⓘ

0%

0%

100%

Need to verify the authenticity of an image?

Try AI image detector